

# Large Language Models – IASD App

Cours 4

April 22, 2026

Shuyu Dong

# Plan du cours

- ▶ 4.1 – Attention, GPT
- ▶ 4.2 – Transformer, GPT
- ▶ 4.2 – TP: GPT from scratch

# Plan du cours

4.1 – Attention, GPT

4.2 – TP: GPT from scratch

5.1 – BERT

5.2 – TP: DistilBERT for text classification

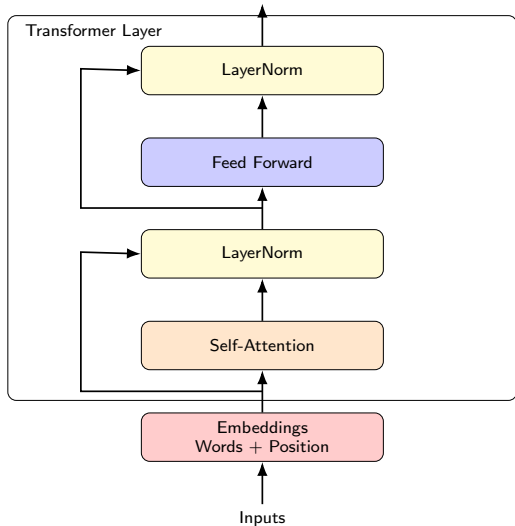
6.1 – Encoder/decoder architectures

6.2 – Fine-tuning, generative models and evaluation of LLMs

7.1 – Scaling LLMs

7.2 – Final project: papers, guidelines

# The transformer model



**Feed forward:**

- $Y = W^{(2)} \phi_{\text{ReLU}}(W^{(1)}Z + b^{(1)}) + b^{(2)}$



**Feed forward input:** Skip-connection(x) + Attention(x)

- $Z = \text{LayerNorm}(x_i + \text{Attention}(x_1, \dots, x_L))$

where the layer normalization is  $\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2}}$  with

$$\mu = \frac{1}{d} \sum_{j=1}^d x_j, \quad \sigma^2 = \frac{1}{d} \sum_{j=1}^d (x_j - \mu)^2.$$



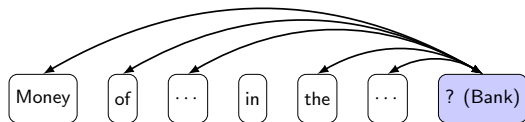
**Self-attention layer:**

- (Cours 3)

# GPT for prediction of next tokens

| Token  | Logits (Top score)               | Prediction | Loss          |
|--------|----------------------------------|------------|---------------|
| [BOS]  | –                                | The        | $-\log(0.6)$  |
| The    | {movie: 0.8, football: 0.6, ...} | movie      | $-\log(0.8)$  |
| movie  | {was: 0.6, is: 0.5, ...}         | was        | $-\log(0.6)$  |
| was    | {nice: 0.6, awful: 0.59, ...}    | nice       | $-\log(0.59)$ |
| aweful | {[END]: 0.9, but: 0.2, ...}      | [END]      | $-\log(0.9)$  |

# GPT for prediction of next tokens

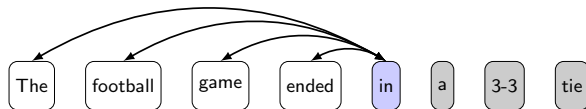


## Remark

Unlike text classification, the input text for training our GPT must be understood as a sequence (with a strictly increasing order). Otherwise, the model training will fail, as the next token can predict itself perfectly, if it is known to the model.

- Position embedding in addition to input words embedding.
- Position embedding map  $P \in \mathbb{R}^{L \times V}$ :  $x_i \mapsto Px_i$  where  $x_i$  is the token id of the word  $w_i$ .

# GPT for prediction of next tokens: causal mask



## Causal mask:

For  $x_i$ , the **causal mask** must be applied to tokens  $x_j$  for  $j \geq i$ .

For  $S = \frac{Q^T K}{\sqrt{d}}$ :

$$S \leftarrow S + M \quad \text{where } M_{ij} = \begin{cases} -\infty & \text{if } j \geq i \\ 0 & \text{if } j < i \end{cases} \quad \text{for all } i.$$

Note that  $\exp(-\infty) = 0$ .

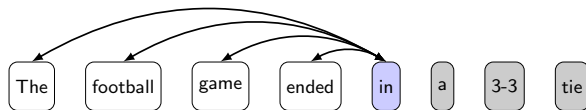
Therefore:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q^T K}{\sqrt{d}} + M\right)V,$$

so that each token pays attention only to precedent tokens in the sequence.

```
1 mask = torch.triu( )
```

# GPT for prediction of next tokens: causal mask



## Causal mask:

Attention( $Q, K, V$ ) =  $\text{softmax}(\frac{Q^T K}{\sqrt{d}} + M)V$ , where  $M_{ij} = \begin{cases} -\infty & \text{if } j \geq i \\ 0 & j < i \end{cases}$  for all  $i$ .

## Why are position embeddings still needed?

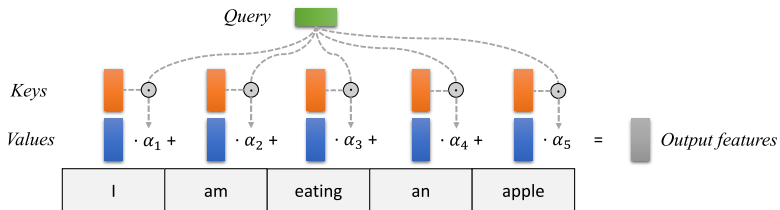
Without position embeddings, the attention  $\alpha_{ij} = \sum_j \text{softmax}(\frac{q_i^T k_j}{\sqrt{d}} + M_{ij})v_j$  would still be a function of the (unordered) set  $\{w_1, \dots, w_{i-1}\}$ , instead of the sequence  $(w_1, \dots, w_{i-1})$ .

## Example:

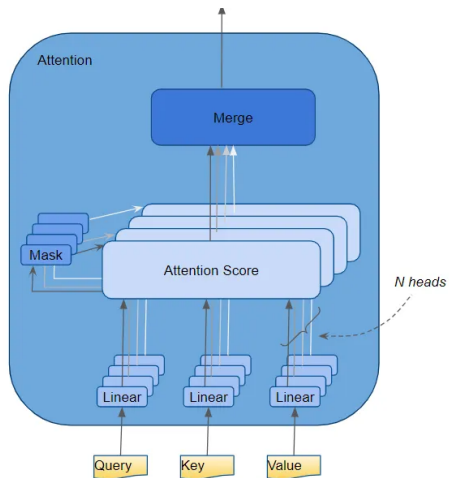
- “A invited B to be his leading actor in the latest movie ...” would be similar to “B invited A to be his leading actor in the latest movie ...” for pre-training.  
(suppose that A and B are both common and frequent words in the training text)



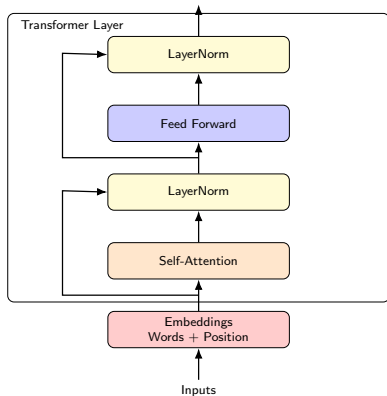
# From attention to multihead attention



## From attention to multihead attention (cont.)



# GPT: training (pre-training)



**Training objectif:**

minimize

$$f_{\theta}(X) = - \sum_i \log P_{\theta}(\hat{y}(x_i) = y_i | x_i)$$

where  $P_{\theta}(\hat{y}(x_i) = y_i | x_i)$  is the output of the GPT.

↪ The objective function is the **cross-entropy** given that  $y_i = x_i$  for the pre-training (self-supervised learning).

**Feed forward:**

$$\bullet Y = W^{(2)} \phi_{\text{ReLU}}(W^{(1)}Z + b^{(1)}) + b^{(2)}$$



**Feed forward input:** Skip-connection(x) + Attention(x)

$$\bullet Z = \text{LayerNorm}(x_i + \text{Attention}(x_1, \dots, x_L))$$



**Self-attention layer:** (Cours 3).

# Inference for next token generation

## Pre-training objectif:

minimize

$$f_{\theta}(X) = - \sum_i \log P_{\theta}(\hat{y}(x_i) = y_i | x_i)$$

where  $P_{\theta}(\hat{y}(x_i) = y_i | x_i)$  is the output of the GPT.

↪ The objective function is the **cross-entropy** given that  $y_i = x_i$  for the pre-training (self-supervised learning).

## Inference after pre-training:

Ideally, we would consider

$$\tilde{y} = \arg \max_y P_{\theta}(y|x) = \arg \max_{y_1, \dots, y_L} \prod_{i=1}^L P_{\theta}(y_i | y_1, \dots, y_{i-1}).$$

But this is **intractable**, since the above decision rule defined over  $V^L$  combinations.

# Inference for next token generation

## Pre-training objectif:

minimize

$$f_{\theta}(X) = - \sum_i \log P_{\theta}(\hat{y}(x_i) = y_i | x_i)$$

where  $P_{\theta}(\hat{y}(x_i) = y_i | x_i)$  is the output of the GPT.

The objective function is the **cross-entropy** given that  $y_i = x_i$  for the pre-training (self-supervised learning).

## Inference after pre-training:

**Greedy** decoding: compute iteratively

1.  $\tilde{y}_1 = \arg \max_y P_{\theta}(y)$
2.  $\tilde{y}_2 = \arg \max_y P_{\theta}(y | \tilde{y}_1)$
3.  $\tilde{y}_3 = \arg \max_y P_{\theta}(y | \tilde{y}_1, \tilde{y}_2)$
4.  $\vdots$
- t  $\tilde{y}_t = \arg \max_y P_{\theta}(y | \tilde{y}_1, \dots, \tilde{y}_{t-1})$

# Plan du cours

4.1 – Attention, GPT

4.2 – TP: GPT from scratch

5.1 – BERT

5.2 – TP: DistilBERT for text classification

6.1 – Encoder/decoder architectures

6.2 – Fine-tuning, generative models and evaluation of LLMs

7.1 – Scaling LLMs

7.2 – Final project: papers, guidelines

# Large Language Models – IASD App

Cours 5

April 23, 2026

Shuyu Dong

# Plan du cours

4.1 – Attention, GPT

4.2 – TP: GPT from scratch

**5.1 – BERT**

5.2 – TP: DistilBERT for text classification

6.1 – Encoder/decoder architectures

6.2 – Fine-tuning, generative models and evaluation of LLMs

7.1 – Scaling LLMs

7.2 – Final project: papers, guidelines

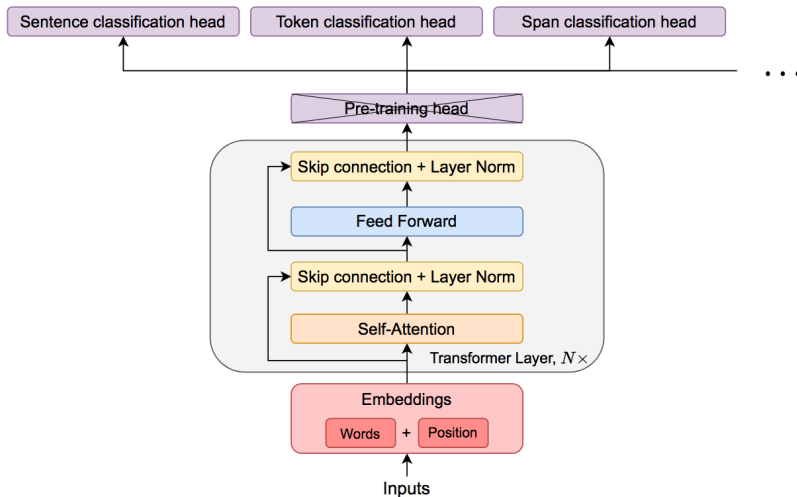


# BERT for text classification

**Pre-training objectif** (for TP4 - DistilBERT for text classification):

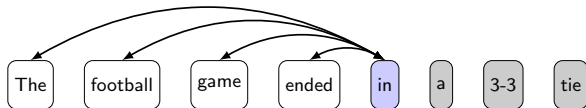
minimize  $f_{\theta}(\mathcal{T}) = -\sum_i \log P_{\theta}(\hat{y}(t^{(i)}) = y^{(i)} | t^{(i)})$  where  $P_{\theta}(\hat{y}(x^{(i)}) = y^{(i)} | t^{(i)})$  is the output of BERT.

The objective function is the **cross-entropy** given that  $y^{(i)}$  is the label of the text  $t^{(i)}$ .

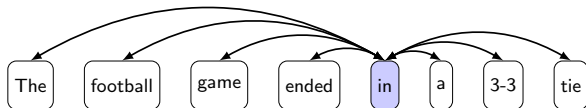


# BERT: encoder architecture

Unlike **Attention for text generation** (with causal mask):



**Attention in BERT:**



$$s_{ij} = \frac{q_i^T k_j}{\sqrt{d}} \text{ for } 1 \leq j \leq i,$$

$$s_{ij} = 0 \text{ for } j > i,$$

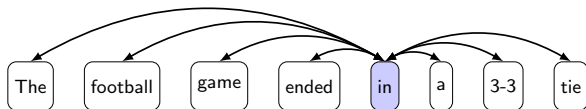
$$y_i = \sum_{j=1}^L \text{softmax}(s_{ij}) v_j.$$

$$s_{ij} = \frac{q_i^T k_j}{\sqrt{d}} \text{ for } 1 \leq j \leq L,$$

$$y_i = \sum_{j=1}^L \text{softmax}(s_{ij}) v_j.$$

# BERT: encoder architecture

## Attention in BERT:



$$s_{ij} = \frac{q_i^T k_j}{\sqrt{d}} \text{ for } 1 \leq j \leq L,$$
$$y_i = \sum_{j=1}^L \text{softmax}(s_{ij}) v_j.$$

## Why is BERT an encoder-only architecture?

BERT was designed not for text generation, but for understanding natural language, such as:

- ▶ Sentiment analysis
- ▶ Text classification
- ▶ Understanding questions

# Plan du cours

4.1 – Attention, GPT

4.2 – TP: GPT from scratch

5.1 – BERT

5.2 – TP: DistilBERT for text classification

6.1 – Encoder/decoder architectures

6.2 – Fine-tuning, generative models and evaluation of LLMs

7.1 – Scaling LLMs

7.2 – Final project: papers, guidelines

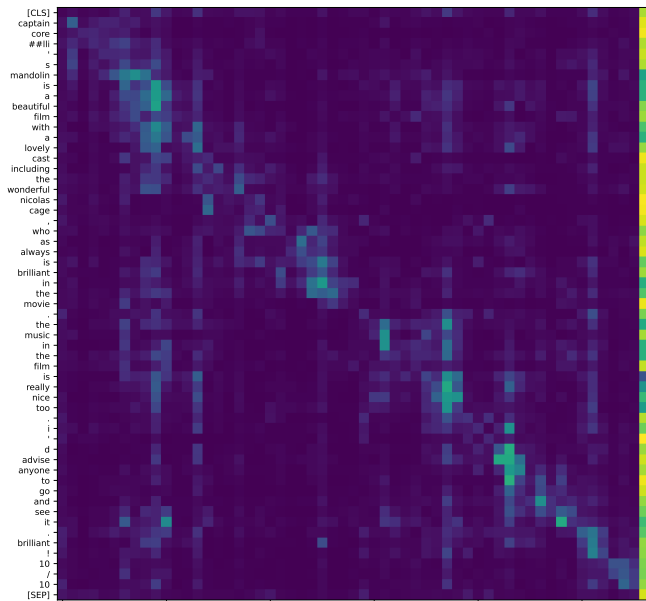
## TP4: BERT for text classification

Reference about Distillation:

(Hinton et al., 2015), Hinton, G., Vinyals, O., and Dean, J. (2015). Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531.

**DistilBERT:** [https://huggingface.co/docs/transformers/model\\_doc/distilbert](https://huggingface.co/docs/transformers/model_doc/distilbert)

## TP4: DistilBert for text classification



# Large Language Models – IASD App

Cours 6

April 29, 2026

Shuyu Dong

# Plan du cours

4.1 – Attention, GPT

4.2 – TP: GPT from scratch

5.1 – BERT

5.2 – TP: DistilBERT for text classification

6.1 – Encoder/decoder architectures

6.2 – Fine-tuning, generative models and evaluation of LLMs

7.1 – Scaling LLMs

7.2 – Final project: papers, guidelines



# Encoder-decoder architectures

An **encoder-decoder** architecture is designed for *conditional generation* tasks:

- ▶ Translation
- ▶ Summarizing an article
- ▶ Answering questions

# Encoder-decoder architectures

An **encoder-decoder** architecture is designed for *conditional generation* tasks:

- ▶ Encoder: trained given full access to all texts  $\mathcal{T}$ .  
**BERT's Pre-training**: Masked Language Modeling (MLM)
  - Mask randomly 15% of the tokens in the input text.
  - Objective: predict (or recover) the masked tokens.
- ▶ Decoder: output  $y$  by text/sequence generation (causal generation):

$$\arg \max P_{\theta}(y_i | y_1, \dots, y_{i-1}, \mathcal{T}).$$

Well-known decoders:

- GPT2 (Radford et al., 2019), GPT3 (Brown et al., 2020)
- LLama (Touvron et al., 2023a), LLama-2 (Touvron et al., 2023b), LLama-3.

# BERT: pre-training

Formulation of **pre-training** via self-supervised learning:

- construct dataset by creating “labels” from un-annotated texts

$$\mathcal{T} : \mathcal{D} = \{t_i\}_i \rightarrow \tilde{\mathcal{D}} = \{\tilde{x}_i, \tilde{y}_i\}_i,$$

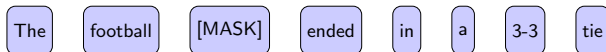
where  $\mathcal{T}$  is a transformation over documents.

**BERT's pre-training:** Masked Language Modeling (MLM)

Original text  $x$ :



Masked input text  $\tilde{x}$ :



The objective function of pre-training is:

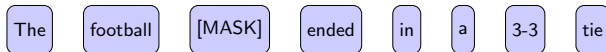
$$\mathcal{L}(\theta) = \sum_{w \text{ is masked}} -\log P_{\theta}(w|\tilde{x}).$$

# BERT Pre-training (task #1): Masked Language Modeling (MLM)

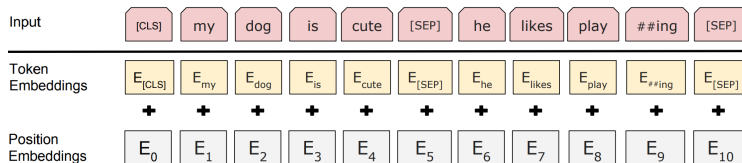
Original text  $x$ :



Masked input text  $\tilde{x}$ :



**Input embeddings:**



(Devlin et al., 2019)

# BERT Pre-training (task #2): Next Sentence Prediction (NSP)

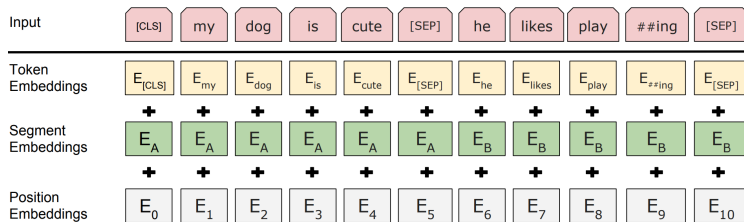
**Goal:** enhance the model's **understanding of sentences** (in addition to token-scale knowledge with MLM).

From each document  $t \in \mathcal{D}$ , extract a sentence denoted as  $s_1$ :

- ▶ With 50% chance, let  $s_2$  be the sentence following  $s_1$  in  $t$ .
- ▶ With 50% chance, let  $s_2$  be a random sentence from  $\mathcal{D}$ .

Define the pre-training objective as

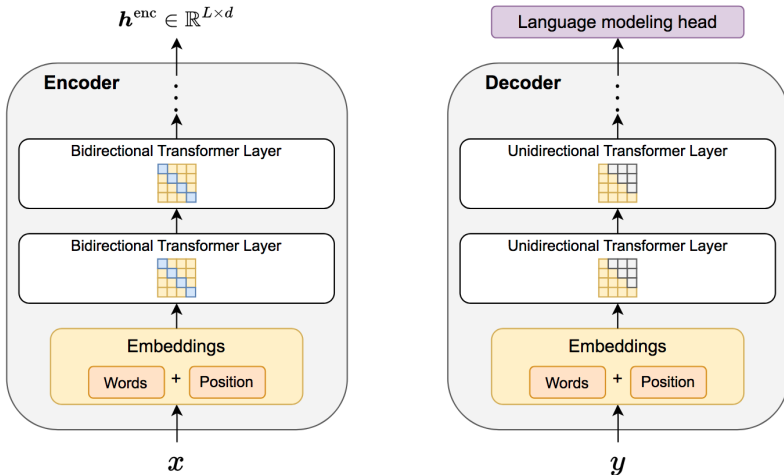
$$\mathcal{L}(\theta) = -\mathbf{1}_{s_2 \text{ follows } s_1} \log P_{\theta}(s_1, s_2) - \mathbf{1}_{s_2 \text{ random}} \log(1 - P_{\theta}(s_1, s_2)).$$



(Devlin et al., 2019)

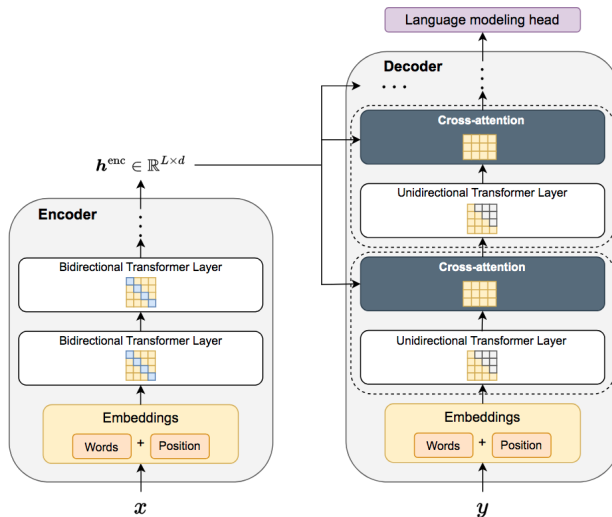
# Encoder-decoder architectures

An **encoder-decoder** architecture is designed for *conditional generation* tasks:



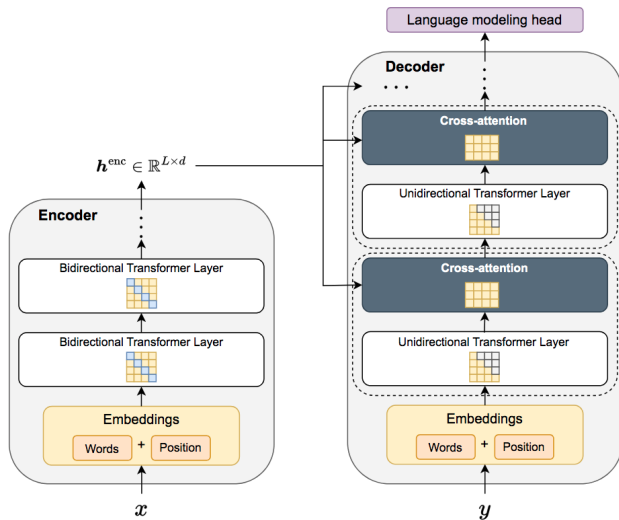
# Encoder-decoder architectures

An **encoder-decoder** architecture is designed for *conditional generation* tasks:



# Encoder-decoder architectures: cross-attention

Contextualization with the encoder output:



$$q_i = Qz_i$$

$$k_j = Kh_j^{\text{enc}}$$

$$v_j = Vh_j^{\text{enc}}$$

$$s_{ij} = q_i^T k_j = z_i^T Q^T K h_j^{\text{enc}}, \quad 1 \leq j \leq L,$$

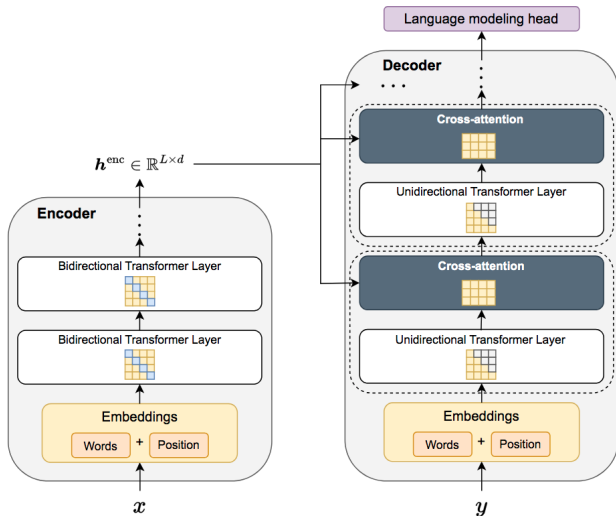
$$\alpha_{ij} = \text{softmax}(s_{ij})$$

$$y_i = \sum_{j=1}^L \alpha_{ij} v_j.$$



# Encoder-decoder architectures: cross-attention

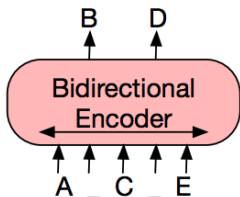
Contextualization with the encoder output:



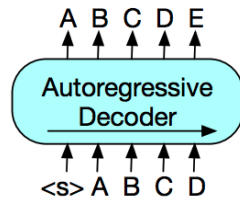
## Applications:

- Text **summarization** (Lewis et al., 2020)  
(pre-training necessary)
- Text generation for **question answering** (Raffel et al., 2020)  
(pre-training necessary)
- Machine **translation** (Vaswani et al., 2017)

# BART



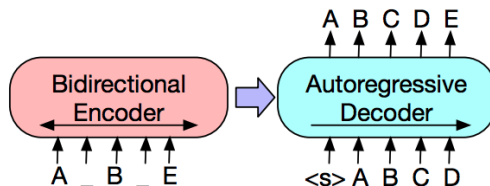
(a) BERT: Random tokens are replaced with masks, and the document is encoded bidirectionally. Missing tokens are predicted independently, so BERT cannot easily be used for generation.



(b) GPT: Tokens are predicted auto-regressively, meaning GPT can be used for generation. However words can only condition on leftward context, so it cannot learn bidirectional interactions.

(Lewis et al., 2020)

# BART



(c) BART: Inputs to the encoder need not be aligned with decoder outputs, allowing arbitrary noise transformations. Here, a document has been corrupted by replacing spans of text with mask symbols. The corrupted document (left) is encoded with a bidirectional model, and then the likelihood of the original document (right) is calculated with an autoregressive decoder. For fine-tuning, an uncorrupted document is input to both the encoder and decoder, and we use representations from the final hidden state of the decoder.

(Lewis et al., 2020)

# Plan du cours

4.1 – Attention, GPT

4.2 – TP: GPT from scratch

5.1 – BERT

5.2 – TP: DistilBERT for text classification

6.1 – Encoder/decoder architectures

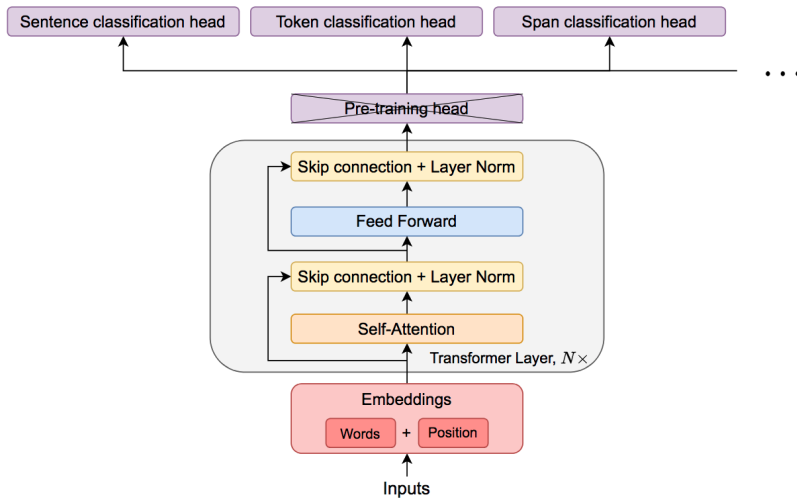
6.2 – Fine-tuning, generative models and evaluation of LLMs

7.1 – Scaling LLMs

7.2 – Final project: papers, guidelines

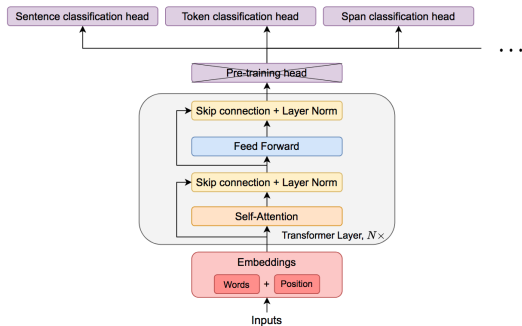
# Fine-tuning

Fine-tuning to specific supervised learning tasks:



(LLM Course 2024)

# Fine-tuning



(LLM Course 2024)

## ► Pre-training:

Training the model on large, generic text data to learn language patterns.

- Training objective: Masked Language Modeling (MLM)

## ► Fine-tuning:

Adapting the pre-trained model to specific tasks by further training on a labeled dataset

Why “Pre-training + Fine-tuning”:

Leverages pre-trained representations tailored for general language-related tasks to more specific applications. Faster and more data-efficient than training from scratch.

# Fine-tuning tasks: sentiment classification

Sentiment classification: a **sequence-level** task.

- ▶ **Text:**  
“The movie was great!”
- ▶ **Model input:**  
[CLS] The movie was great [SEP]
- ▶ **Output:**  
Prediction of the class among {positive, negative}.

(Seen in TP4–DistilBert)

**Datasets:** IMDB, SST-2

# Fine-tuning tasks: natural language inference (NLI)

Classification of sentence pairs as entailment, contradiction, or neutral: a **sequence-level** task.

- ▶ **Text:**

Sentence 1: "A person is playing guitar."

Sentence 2: "The man is a singer"

Label: Neutral

- ▶ **Model input:**

[CLS] A person is playing guitar [SEP] The man is a singer [SEP]

- ▶ **Output:**

Prediction of the class among {Entailment, Contradiction, Neutral}.

**Datasets:** MNLI



# Fine-tuning tasks: natural language inference (NLI)

Identification of entities: a **token-level** task.

- ▶ **Text:**

Text: "Apple was founded by Steve Jobs in Cupertino."

Labels: Apple (B-ORG), was (O), founded (O), by (O), Steve (B-PER), Jobs (I-PER), in (O), Cupertino (B-LOC)

- ▶ **Model input:**

[CLS] Apple was founded by Steve Jobs in Cupertino [SEP]

- ▶ **Output:**

Prediction of token labels: B-ORG, I-ORG, B-PER, I-PER, B-LOC, I-LOC

**Datasets:** CoNLL-2003

# Fine-tuning tasks: machine translation

Translation of sentences from EN to FR.

## Training:

### ► Text:

Text: "The cat is sleeping on the couch."

Target: "Le chat dort sur le canapé."

### ► Model input:

[BOS] Translate to French: The cat is sleeping on the couch. Le chat dort sur le canapé

### ► Output:

Translate to French: The cat is sleeping on the couch. Le chat dort sur le canapé [EOS]

**Objective:** train the model to predict the next tokens given the input/output samples.

# Fine-tuning tasks: machine translation

Translation of sentences from EN to FR.

## **Inference:**

### ► **Text:**

Text: "(Translate to French) The sun is shining."

### ► **Model input:**

[BOS] Translate to French: The sun is shining.

### ► **Output:**

Le soleil brille [EOS]

The model generates the translation autoregressively, conditioned on the Model Input until reaching [EOS].

# Fine-tuning tasks: summary

## Text classification:

- ▶ Sentiment analysis
- ▶ Spam detection
- ▶ Topic classification

## Sequence pairs classification:

- ▶ Natural Language Inference (NLI)
- ▶ Paraphrase Detection
- ▶ Stance Detection

## Token classification:

- ▶ Named Entity Recognition (NER)
- ▶ Part-of-Speech (POS) Tagging
- ▶ Chunking (Phrase Extraction)

## Examples of Generative Tasks:

- ▶ Text Summarization: Generate a concise summary of a longer document.

## Fine-tuning tasks: summary (cont.)

- ▶ Machine Translation: Translate text from one language to another.
- ▶ Dialogue Generation: Produce conversational responses in chatbots.
- ▶ Code Generation: Generate programming code based on natural language descriptions.
- ▶ Data-to-Text Generation: Convert structured data (e.g., tables) into natural language descriptions.
- ▶ Paraphrasing: Rephrase text while preserving its meaning.
- ▶ Question Answering: Generate answers to questions based on a given context.

# Large Language Models – IASD App

Cours 7

April 30, 2026

Shuyu Dong

# Plan du cours

4.1 – Attention, GPT

4.2 – TP: GPT from scratch

5.1 – BERT

5.2 – TP: DistilBERT for text classification

6.1 – Encoder/decoder architectures

6.2 – Fine-tuning, generative models and evaluation of LLMs

**7.1 – Scaling LLMs**

7.2 – Final project: papers, guidelines

# Large models

- ▶ BERT-like models achieve better performances than humans on the SuperGLUE dataset.
- ▶ BART-like models can summarize texts, answer questions, etc.
- ▶ GPT3-3.5-4 are few-shot learners, they can answer questions based on their general knowledge.
- ▶ ChatGPT.

## **How did these model achieve such performance?**

- ▶ Huge amount of training data.
- ▶ Scaling the models.



# Large models

- ▶ BERT-like models achieve better performances than humans on the SuperGLUE dataset.
- ▶ BART-like models can summarize texts, answer questions, etc.
- ▶ GPT3-3.5-4 are few-shot learners, they can answer questions based on their general knowledge.
- ▶ ChatGPT.

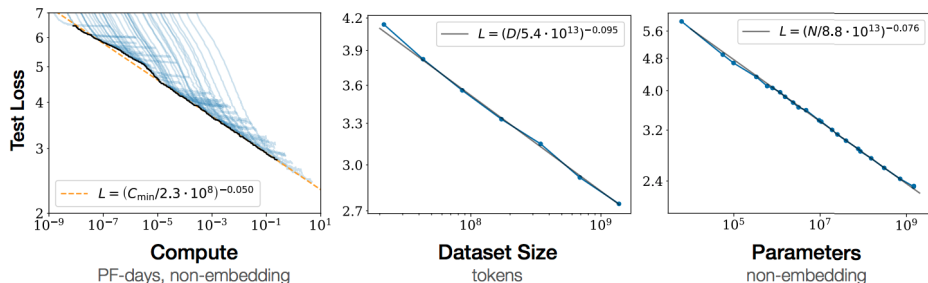
## How did these model achieve such performance?

- ▶ Data.
- ▶ Scaling the models.

| Model  | Params | Context | Batch | Steps |
|--------|--------|---------|-------|-------|
| BERT   | 355M   | 512     | 256   | 1M    |
| BART   | 406M   | 1024    | 8000  | 500K  |
| Llama2 | 7B–70B | 4096    | 4000  | 500K  |

# Large models: the scaling laws

Up till now, the LLMs' performances have followed the scaling laws:



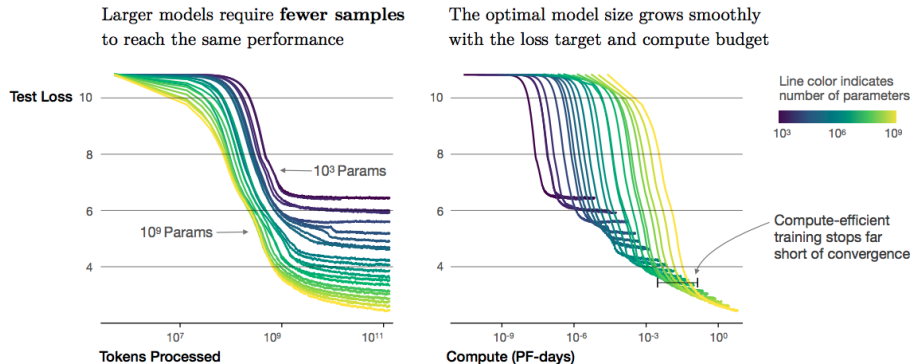
**Figure 1** Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute<sup>2</sup> used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

(Kaplan et al., 2020)

(Kaplan et al., 2020) Kaplan et al., 2020. Scaling laws for neural language models. arXiv preprint arXiv:2001.08361.

# Large models: the scaling laws

Up till now, the LLMs' performances have followed the scaling laws:



**Figure 2** We show a series of language model training runs, with models ranging in size from  $10^3$  to  $10^9$  parameters (excluding embeddings).

(Kaplan et al., 2020)

(Kaplan et al., 2020) Kaplan et al., 2020. Scaling laws for neural language models. arXiv preprint arXiv:2001.08361.

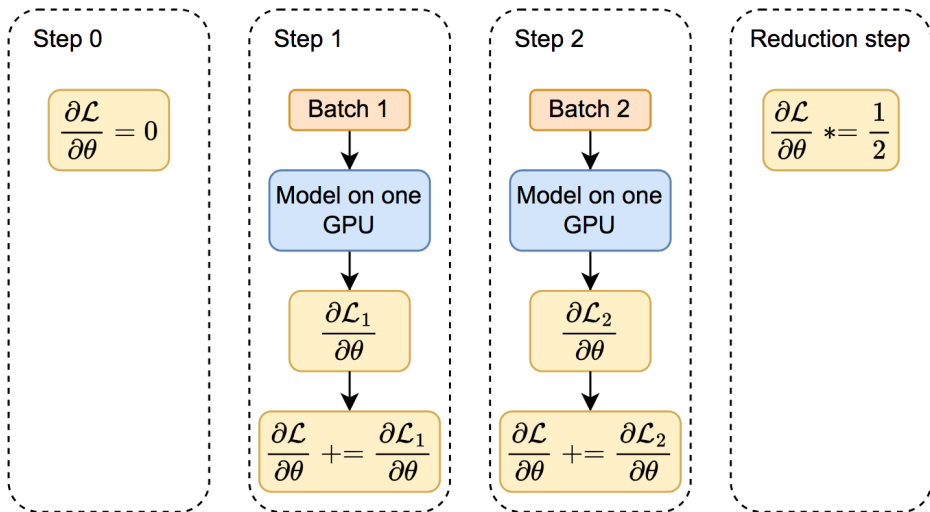
# Large models: pre-training in practice

- ▶ Biggest NVIDIA GPUs are  $\geq 8\times$  bigger than Colab GPUs.
- ▶ Even with bigger GPUs, processing batches of 8000 documents is an issue.

How can we do that in practice?

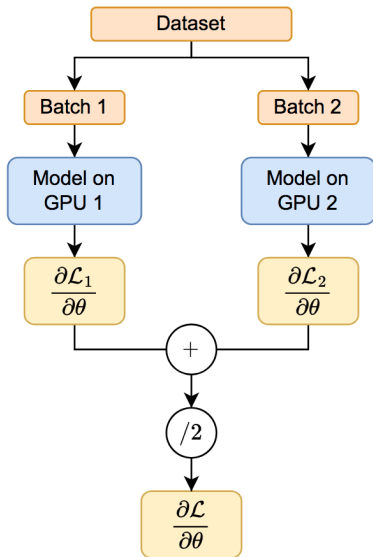
⇒ Review methods for heavy training workloads.

## Scaling data: gradient accumulation



(LLM Course 2024)

## Scaling data: data parallelism



(LLM Course 2024)

# Scaling data: data parallelism

Gradient accumulation and data parallelism increase the effective batch size.

## Gradient Accumulation

| Batch Size | Accumulation Steps | Effective Batch Size |
|------------|--------------------|----------------------|
| 32         | 1                  | 32                   |
| 32         | 2                  | 64                   |
| 32         | 4                  | 128                  |

## Data Parallelism

| Per Device Batch Size | Number of Devices | Effective Batch Size |
|-----------------------|-------------------|----------------------|
| 32                    | 1                 | 32                   |
| 32                    | 2                 | 64                   |
| 32                    | 4                 | 128                  |

## Scaling data: gradient accumulation and data parallelism

|              | <b>Gradient Accumulation</b> | <b>Data Parallelism</b>               |
|--------------|------------------------------|---------------------------------------|
| Parallelism  | No, sequential               | Yes, across GPUs                      |
| Several GPUs | Optional                     | Yes                                   |
| Time         | Slower                       | Faster (beware of communication time) |

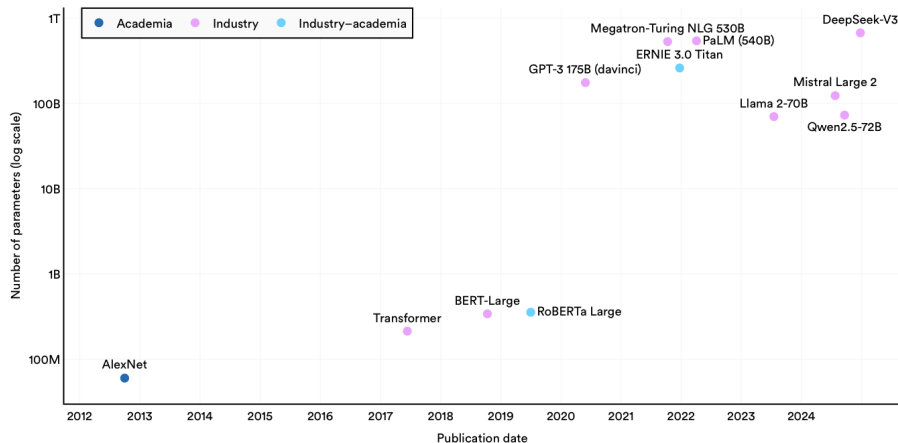
- ▶ Data parallelism is useful with several GPUs.
- ▶ Gradient accumulation is useful with limited resources.
- ▶ Both can be combined.



# Scaling the models

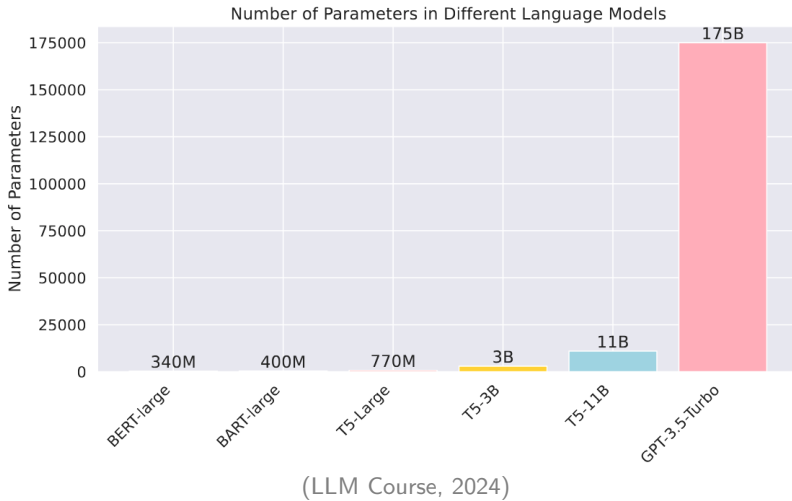
## Number of parameters of select notable AI models by sector, 2012–24

Source: Epoch AI, 2025 | Chart: 2025 AI Index report



Source: <https://hai.stanford.edu/ai-index/2025-ai-index-report>

# Scaling the models



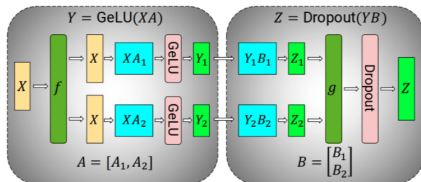
# Scaling the models

- ▶ GPT2-Large on a GPU is  $\sim 3.5$  GiB.
- ▶ GPT-3 is approximately  $250\times$  larger.
- ▶ Biggest GPUs are 80 GiB.

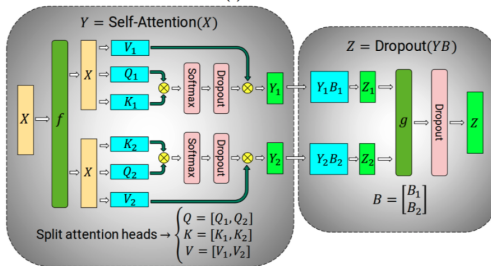
## How does it fit?

We discuss training and inference.

# Scaling the models: model parallelism



(a) MLP.



(b) Self-Attention.

(Shoeybi et al., 2019), (LLM Course, 2024)

(Shoeybi et al., 2019) Megatron-LM: Training multi-billion parameter language models using model parallelism.

# Scaling the models: pre-training

- ▶ Heavy hardware optimizations are required.
- ▶ Independent costly operations are split across devices.
- ▶ Final results are aggregated.

⇒ Model parallelism is difficult in practice.

# Fine-tuning/inference from large models for end-users?

For users who want to use these large models for:

- ▶ inference
- ▶ fine-tuning on a specific task

**What solutions are there?**

# Fine-tuning/inference from large models: Quantization

GPUs standard precision is float32. Quantization reduces precision.

| Data Type | Bit Width | Hardware         | Training |
|-----------|-----------|------------------|----------|
| float32   | 32        | CPUs/GPUs        | Yes      |
| float16   | 16        | FP16 GPUs        | Yes      |
| bfloat16  | 16        | Ampere GPUs/TPUs | Yes      |
| int8      | 8         | CPUs/GPUs        | No       |

- ▶ Empirically: the models often maintain performance.
- ▶ Check hardware support and instabilities.
  - Example: T5 does not work with float16.

# Reduce the cost of autodiff

| Component        | Memory Cost     | Inference |
|------------------|-----------------|-----------|
| Model Parameters | $O(P)$          | Yes       |
| Activations      | $O(B \times L)$ | Yes/No    |
| Gradients        | $O(P)$          | No        |
| Optimizer States | $O(P)$          | No        |

Huge dependency on the number of parameters.

**Idea:** Reduce the number of parameters to update!



# Training large models: parameter-efficient fine-tuning

Let  $\theta \in \mathbb{R}^P$  be the parameters of a base model.

The goal of **parameter efficient finetuning (PEFT)** is to select a subset of parameters

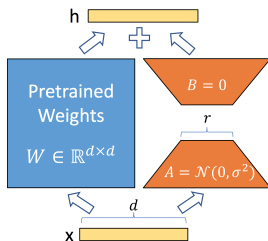
$$\theta' \in \mathbb{R}^{P'}$$

with  $P' \ll P$  while maintaining performance.

(See TP with TinyLLama)

Example: Train only the last layer.

**Related method:** Low-rank adaptation, LoRA (Hu et al., 2022)



(Hu et al., 2022) LoRA: Low-rank adaptation of large language models. ICLR 2022.

# Training large models: scaling the context length

## Limitations:

- ▶ architectural limitations: the context length  $L$  needs to be fixed, with position embeddings (of fixed size  $\mathbb{R}^{L \times d}$ )
- ▶ training setup limitations: we need to set up a maximal context length, as long as possible
- ▶ computational cost: the cost grows with the context length  $L$

# Training large models: scaling the context length

Limitations:

- ▶ architectural limitations: the context length  $L$  needs to be fixed, with position embeddings (of fixed size  $\mathbb{R}^{L \times d}$ )
- ▶ training setup limitations: we need to set up a maximal context length, as long as possible
- ▶ computational cost: the cost grows with the context length  $L$

**Remark:** Attention computation is  $O(L^2)$ .

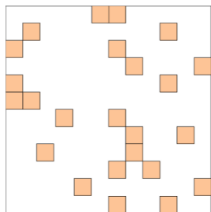
Let  $L$  be the input length.

$$QK^T \in \mathbb{R}^{L \times L}.$$

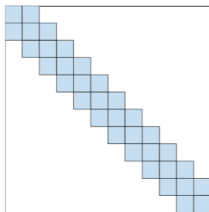
$\rightsquigarrow$  Computational cost for the attention matrix is quadratic with respect to  $L$ .

**How can we reduce that?**

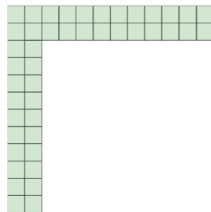
# Scaling the context length: sparse attention



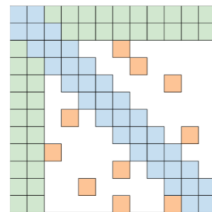
(a) Random attention



(b) Window attention



(c) Global Attention



(d) BIGBIRD

(Zaheer et al., 2020)

- ▶ LongT5 (Guo et al., 2021)
- ▶ BigBird (Zaheer et al., 2020)
- ▶ Longformer (Beltagy et al., 2020)

(Zaheer et al., 2020) Big bird: Transformers for longer sequences. Advances in neural information processing systems, 33, pp.17283-17297.

# Scaling the context length: other solutions

## Hardware improvements

- ▶ FlashAttention (Dao et al., 2022) provides optimized CUDA kernels.

## Stick to the model's base length

- ▶ 4k tokens is already large.
- ▶ LLMs do not use their full context (Ravaut et al., 2023)
- ▶ Use retrieval techniques.
- ▶ Process documents by chunks.

## New architectures are being proposed:

- ▶ State-space models
- ▶ MAMBA

# Plan du cours

4.1 – Attention, GPT

4.2 – TP: GPT from scratch

5.1 – BERT

5.2 – TP: DistilBERT for text classification

6.1 – Encoder/decoder architectures

6.2 – Fine-tuning, generative models and evaluation of LLMs

7.1 – Scaling LLMs

7.2 – Final project: papers, guidelines

# Final Project

## Guidelines

Each team of at most 4 people should **select one article** from the provided list to read and analyze. The goal is to create a clear, concise presentation based on the article. Up to 2 teams can select the same paper.

Additionally, each team should **implement an algorithm** described in the article and conduct a small experiment. The experiment should be rather short and involve creating one or two plots to explore specific experimental dimensions.

The expected deliverables are an **oral presentation** (10min) and a **report**. The report is up to 6 pages, but less than 6 is fine and won't be penalized as long as it is enough to cover what you did.

**Mandatory:** disclose your usage of AI Assistants in a dedicated section.

Below are some questions you may use to guide your reading (among others):

- ▶ Did you like the paper? Did you find it interesting?
- ▶ What are the most important things you learned from the paper? Why are they important?
- ▶ Do the lessons learned generalize beyond the specific task? Do they contribute towards building an important system or application?

# Final Project (cont.)

- ▶ Is the experimental setup satisfying? Any experiments missing? Any obvious or important baseline missing?
- ▶ Is the problem/approach well motivated?
- ▶ Are you convinced by the results? Why?
- ▶ Is the writing clear? Is the paper well structured?

## Registration

Registration form for the group information and the choice of the article to read: [link here](#).

The **complete information** about the project and its submission is available at <https://github.com/shuyu-d/llms2025-2026/blob/main/projet-final.md>.



Thank You !

## References

- Beltagy, I., Peters, M. E., and Cohan, A. (2020). Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186.
- Guo, M. et al. (2021). Longt5: efficient text-to-text transformer for long sequences. *arxiv preprint arXiv:2112.07916*.
- Hinton, G., Vinyals, O., and Dean, J. (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. (2022). Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3.

## References (cont.)

- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.
- Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., and Zettlemoyer, L. (2020). Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 7871–7880.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. (2019). Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.
- Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67.
- Ravaut, M., Joty, S., Sun, A., and Chen, N. F. (2023). On context utilization in summarization with large language models. *arxiv e-prints*, pages arxiv–2310.
- Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., and Catanzaro, B. (2019). Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*.

## References (cont.)

- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. (2023a). Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. (2023b). Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Zaheer, M., Guruganesh, G., Dubey, K. A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., et al. (2020). Big bird: Transformers for longer sequences. *Advances in neural information processing systems*, 33:17283–17297.